

Advanced Machine Learning

Lecture 8: Recurrent Neural Networks

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

3 December 2024



Outline

Repetition

- Understanding CNN
- Adversarial attacks
- Adversarial training & XAI
- Attacks on humans

Recurrent Neural Networks

- Motivation
- Cells & layers
- General definition
- Backpropagation & training

Outline

Repetition

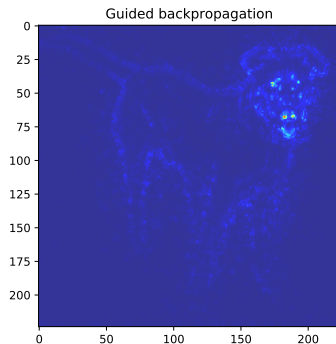
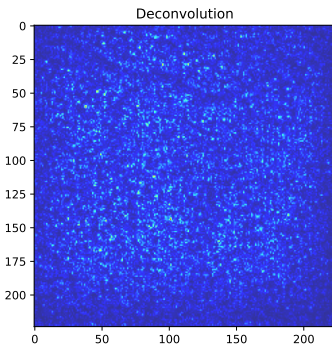
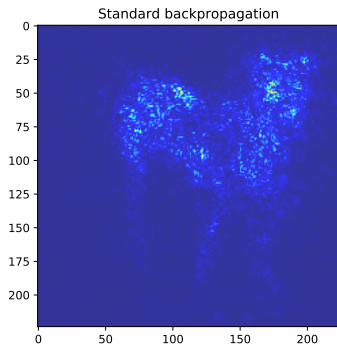
- Understanding CNN
- Adversarial attacks
- Adversarial training & XAI
- Attacks on humans

Recurrent Neural Networks

- Motivation
- Cells & layers
- General definition
- Backpropagation & training

Attribution: deconvolution & (guided) backpropagation

Example of attribution: **saliency maps** w/ **three approaches**.

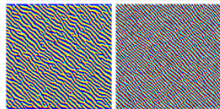
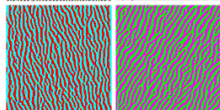
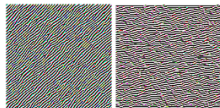


<https://www.flickr.com/photos/jmorgan/5164287/>, scaled to 224×224 , CC BY-SA 2.0

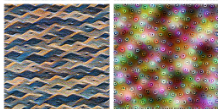
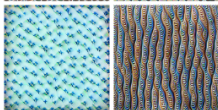
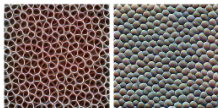
Deconvolution: often highly oscillating **artifacts**, **guided backpropagation**: **better** results.

Feature visualization

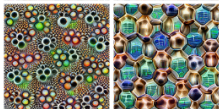
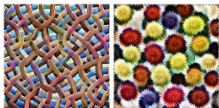
Gradient ascent (w/ regularization & transformations) to find **extremal activation**, see Simonyan, Vedaldi, and Zisserman [8] and Olah, Mordvintsev, and Schubert [6].



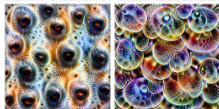
Edges (layer conv2d0)



Textures (layer mixed3a)



Patterns (layer mixed4a)



Parts (layers mixed4b & mixed4c)

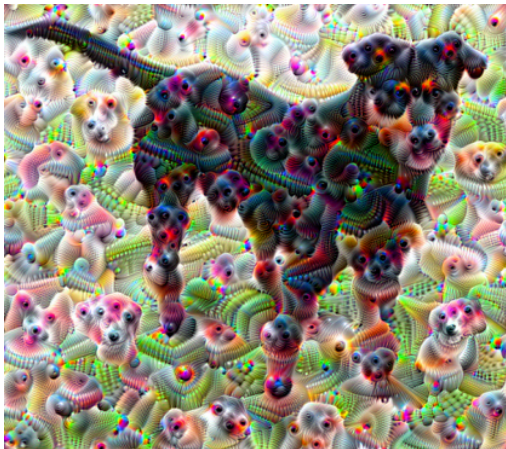


Objects (layers mixed4d & mixed4e)

Image taken from Olah, Mordvintsev, and Schubert [6]

Deep Dream

Deep dream: InceptionV3, mixed3 + mixed5



Deep dream: InceptionV3, mixed2 + mixed10



<https://www.flickr.com/photos/jmorgan/5164287/>, CC BY-SA 2.0

Fooling CNN

 x

“panda”

57.7% confidence

 $+ .007 \times$  $\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

 $=$  $x +$ $\epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

Taken from Goodfellow, Shlens, and Szegedy [4], available at <http://www.cleverhans.io/security/privacy/ml/2016/12/16/breaking-things-is-easy.html>.

How to find adversarial examples

Given: neural net for classification $f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$, data $\mathbf{x} \in \mathbb{R}^n$ with correct class $c \approx f(\mathbf{x})$, **seek** “close” $\mathbf{x} + \mathbf{r}$ such that

$$f(\mathbf{x} + \mathbf{r}) \approx w \neq c.$$

Box-constrained optimization problem (pixel values between 0 and 1):

$$\min_{\|\mathbf{r}\|_2} \quad \text{s.t.} \quad f(\mathbf{x} + \mathbf{r}) = w \quad \wedge \quad \mathbf{x} + \mathbf{r} \in [0, 1]^n.$$

The authors use optimization based on a **cost function** $C(\mathbf{x}, c)$ that measures the distance of the output $f(\mathbf{x})$ of the neural net to the class c . They pose the **line-search optimization problem**

$$\min (\alpha \|\mathbf{r}\|_2 + C(\mathbf{x} + \mathbf{r}, w)) \quad \text{s.t.} \quad \mathbf{x} + \mathbf{r} \in [0, 1]^n,$$

and minimize over all $\alpha > 0$ for which the minimizer $\mathbf{r}$ satisfies $f(\mathbf{x} + \mathbf{r}) = w$.

A remark on the optimization

In Szegedy et al. [9] **box-constrained L-BFGS** is used. **BFGS** is a variant of Newton's method

$$\mathbf{p}_k = -(\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{p}_k \alpha_k,$$

where the Hessian $\nabla^2 f(\mathbf{x}_k)$ is replaced by a **simpler approximation** $\mathbf{B}_k$, i.e., $\mathbf{B}_k \mathbf{p}_k = -\nabla f(\mathbf{x}_k)$.

Broyden [1], Fletcher [2], Goldfarb [3], and Shanno [7] developed this **quasi-Newton scheme**, where the Hessian of the objective function is not computed, but the following **secant condition** is imposed on the next approximate Hessian,

$$\mathbf{B}_{k+1} \mathbf{s}_k = \mathbf{y}_k, \quad \mathbf{s}_k := \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \mathbf{y}_k := \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k),$$

and the following **symmetric rank-two update** is used,

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} - \frac{\mathbf{B}_k \mathbf{s}_k \mathbf{s}_k^\top \mathbf{B}_k^\top}{\mathbf{s}_k^\top \mathbf{B}_k \mathbf{s}_k}, \quad \text{e.g.,} \quad \mathbf{B}_0 = \mathbf{I}.$$

A remark on the optimization

In Szegedy et al. [9] **box-constrained L-BFGS** is used. **L-BFGS** stands for **Limited-memory** BFGS, which was developed for larger optimization problems where the approximate Hessian is too large to be stored.

The update of the **inverse Hessian** $\mathbf{H}_k := \mathbf{B}_k^{-1}$,

$$\mathbf{H}_{k+1} = \left(\mathbf{I} - \frac{\mathbf{s}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} \right) \mathbf{H}_k \left(\mathbf{I} - \frac{\mathbf{y}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k},$$

is obtained using the **Sherman-Morrison-Woodbury formula** ($\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{U} \in \mathbb{R}^{n \times m}$, $\mathbf{V} \in \mathbb{R}^{m \times n}$),

$$(\mathbf{A} + \mathbf{UV})^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1} \mathbf{U} (\mathbf{I}_m + \mathbf{VA}^{-1} \mathbf{U})^{-1} \mathbf{VA}^{-1}.$$

The update $\mathbf{p}_k = -\mathbf{H}_k \mathbf{g}_k$ can be computed **using only** $\mathbf{g}_k$, $\mathbf{s}_j$, $\mathbf{y}_j$, $\rho_j := 1/\mathbf{y}_j^\top \mathbf{s}_j$, $j = 1, \dots, k-1$ and $\mathbf{H}_0 = \mathbf{I}$. In L-BFGS, the application of the inverse is limited to **the last vectors**, see Nocedal [5].

A remark on the optimization

A possible algorithm for the computation of $\mathbf{p}_k \approx -\mathbf{H}_k \mathbf{g}_k$ in L-BFGS looks as follows:

Algorithm Computation of the update in L-BFGS

```
1:  $\mathbf{p} \leftarrow -\mathbf{g}_k$ ;  
2: for  $j = k - 1 : k - m : -1$  do  
3:    $\alpha_j \leftarrow \rho_j \mathbf{s}_j^\top \mathbf{p}$ ;  
4:    $\mathbf{p} \leftarrow \mathbf{p} - \mathbf{y}_j \alpha_j$ ;  
5: end for  
6: if  $k > 0$  then  
7:    $\mathbf{p} \leftarrow \mathbf{p} (\mathbf{s}_{k-1}^\top \mathbf{y}_{k-1}) / (\mathbf{y}_{k-1}^\top \mathbf{y}_{k-1})$ ;  
8: end if  
9: for  $j = k - m : k - 1$  do  
10:   $\alpha_j \leftarrow \alpha_j - \rho_j \mathbf{y}_j^\top \mathbf{p}$ ;  
11:   $\mathbf{p} \leftarrow \mathbf{p} + \mathbf{s}_j \alpha_j$ ;  
12: end for
```

In BFGS and L-BFGS, the curvature condition $\mathbf{s}_k^\top \mathbf{y}_k > 0$ has to be checked.

A simplified approach

Box-constrained L-BFGS freezes the variables that are at the constraints and uses L-BFGS only for the other variables. Sounds complicated. Can we do simpler?

In the example of the panda, Goodfellow, Shlens, and Szegedy [4] used the observation that the **classifications regions** computed by a typical CNN with ReLU activations mostly have **linear borders** and are **linear on the outside**.

The task is to **change the value of the cost function maximally**. The authors use as perturbation η a **multiple of the sign of the gradient**,

$$\eta = \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} C(\mathbf{x}, \mathbf{p}, y)).$$

The gradient can be computed efficiently using **backpropagation**.

This method is known as the **fast gradient sign method**.

Adversarial training & XAI

Adversarial training is a technique to enhance the **robustness** of a CNN to adversarial attacks.

During training, adversarial examples are **generated** based on some of the aforementioned approaches. These are used as **training examples**.

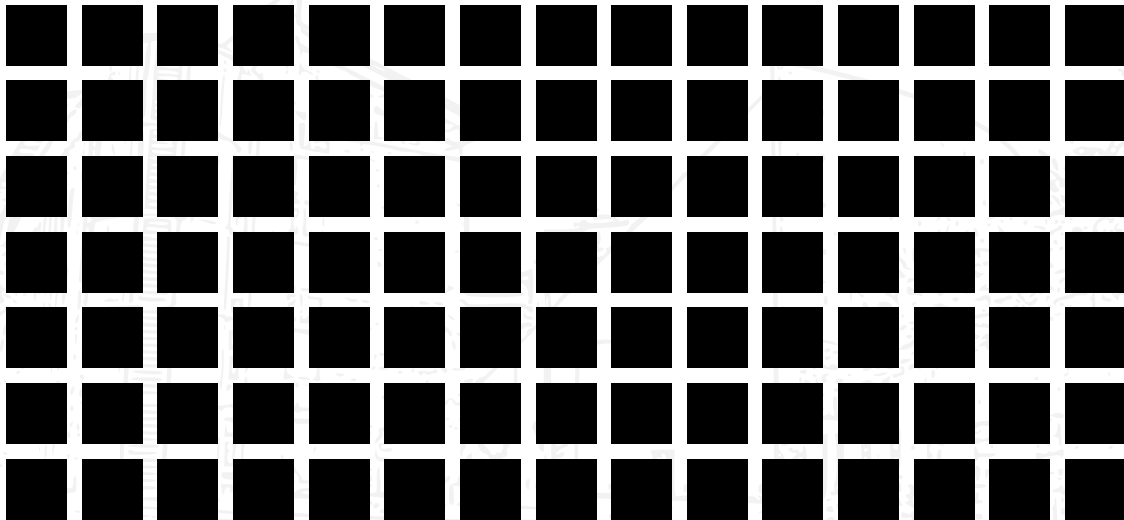
- ▶ This **increases the accuracy** on the test set,
- ▶ The new CNN **still has adversarial examples**, even though simple attacks such as the one pixel attack will typically fail.

Adversarial training **can easily be incorporated** into on the fly construction of new training data, e.g., by slight rotations, distortions, flipping of sides.

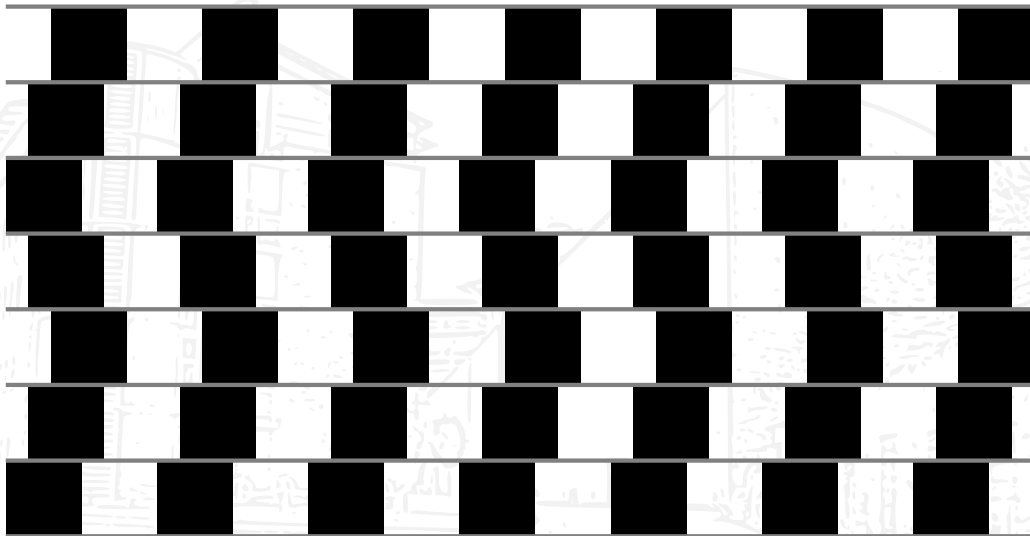
Adversarial examples show the need for a better understanding of CNN. Who is responsible if your **autonomous car caused an accident**?

For such reasons, **eXplainable AI** (XAI) has been developed. Here, the decision of the neural net has to be **accompanied by an explanation understandable to humans**.

Adversarial attacks on humans



Adversarial attacks on humans



Outline

Repetition

- Understanding CNN
- Adversarial attacks
- Adversarial training & XAI
- Attacks on humans

Recurrent Neural Networks

- Motivation
- Cells & layers
- General definition
- Backpropagation & training

Motivation

All network structures we considered so far are constrained to a **fixed input** and **fixed output size**.



Motivation

All network structures we considered so far are constrained to a **fixed input** and **fixed output size**.

This poses problems for **sequences of data**, e.g., **text**, **speech**, **video**:

- ▶ the input may be **sequences of arbitrary lengths** (e.g., text analysis, video frames)
- ▶ the output may be **sequences of arbitrary lengths** (translation of spoken or written language)

Motivation

All network structures we considered so far are constrained to a **fixed input** and **fixed output size**.

This poses problems for **sequences of data**, e.g., **text**, **speech**, **video**:

- ▶ the input may be **sequences of arbitrary lengths** (e.g., text analysis, video frames)
- ▶ the output may be **sequences of arbitrary lengths** (translation of spoken or written language)

Moreover, the **amount of computation** in the network structures considered thus far does **not depend on the input**. Here, an interesting application would be to **process data** in a **sequential manner** by some policy depending on the input.

Motivation

All network structures we considered so far are constrained to a **fixed input** and **fixed output size**.

This poses problems for **sequences of data**, e.g., **text**, **speech**, **video**:

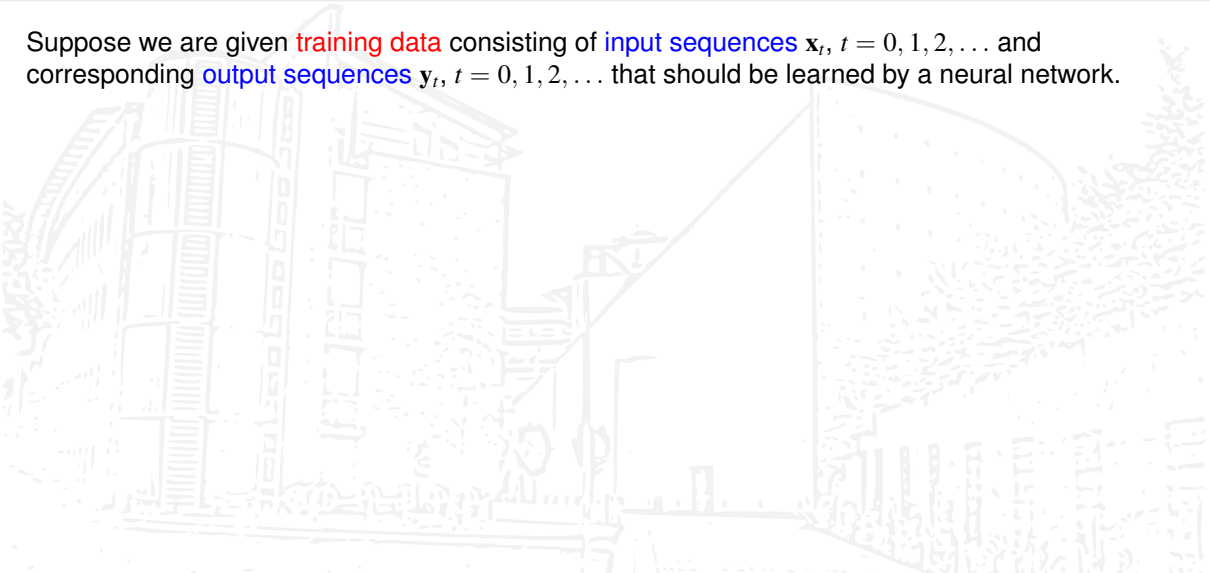
- ▶ the input may be **sequences of arbitrary lengths** (e.g., text analysis, video frames)
- ▶ the output may be **sequences of arbitrary lengths** (translation of spoken or written language)

Moreover, the **amount of computation** in the network structures considered thus far does **not depend on the input**. Here, an interesting application would be to **process data** in a **sequential manner** by some policy depending on the input.

The **solution** to these problems are so-called **Recurrent Neural Networks (RNN)**.

Definition of an RNN cell

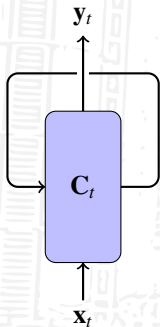
Suppose we are given **training data** consisting of **input sequences** $\mathbf{x}_t, t = 0, 1, 2, \dots$ and corresponding **output sequences** $\mathbf{y}_t, t = 0, 1, 2, \dots$ that should be learned by a neural network.



Definition of an RNN cell

Suppose we are given **training data** consisting of **input sequences** $\mathbf{x}_t, t = 0, 1, 2, \dots$ and corresponding **output sequences** $\mathbf{y}_t, t = 0, 1, 2, \dots$ that should be learned by a neural network.

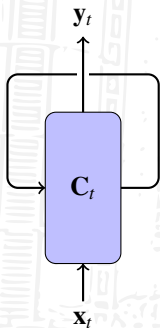
An **RNN cell** for this problem looks as follows:



Definition of an RNN cell

Suppose we are given **training data** consisting of **input sequences** $\mathbf{x}_t, t = 0, 1, 2, \dots$ and corresponding **output sequences** $\mathbf{y}_t, t = 0, 1, 2, \dots$ that should be learned by a neural network.

An **RNN cell** for this problem looks as follows:

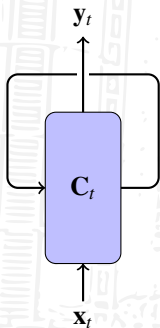


It has a **cell state** $\mathbf{C}_t, t = 0, 1, \dots$ and an **additional output** that is **used** in the **next step** as **additional input**.

Definition of an RNN cell

Suppose we are given **training data** consisting of **input sequences** $\mathbf{x}_t, t = 0, 1, 2, \dots$ and corresponding **output sequences** $\mathbf{y}_t, t = 0, 1, 2, \dots$ that should be learned by a neural network.

An **RNN cell** for this problem looks as follows:



It has a **cell state** $\mathbf{C}_t, t = 0, 1, \dots$ and an **additional output** that is **used** in the **next step as additional input**.

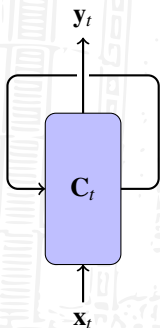
In **most applications**, the cell state is **internally completely described by**:

- ▶ **fixed data**, e.g., some fixed matrices, and
- ▶ the **variable update vector** that is handed to the next iteration, the **hidden state**.

Definition of an RNN cell

Suppose we are given **training data** consisting of **input sequences** $\mathbf{x}_t$, $t = 0, 1, 2, \dots$ and corresponding **output sequences** $\mathbf{y}_t$, $t = 0, 1, 2, \dots$ that should be learned by a neural network.

An **RNN cell** for this problem looks as follows:



It has a **cell state** C_t , $t = 0, 1, \dots$ and an **additional output** that is **used** in the **next step as additional input**.

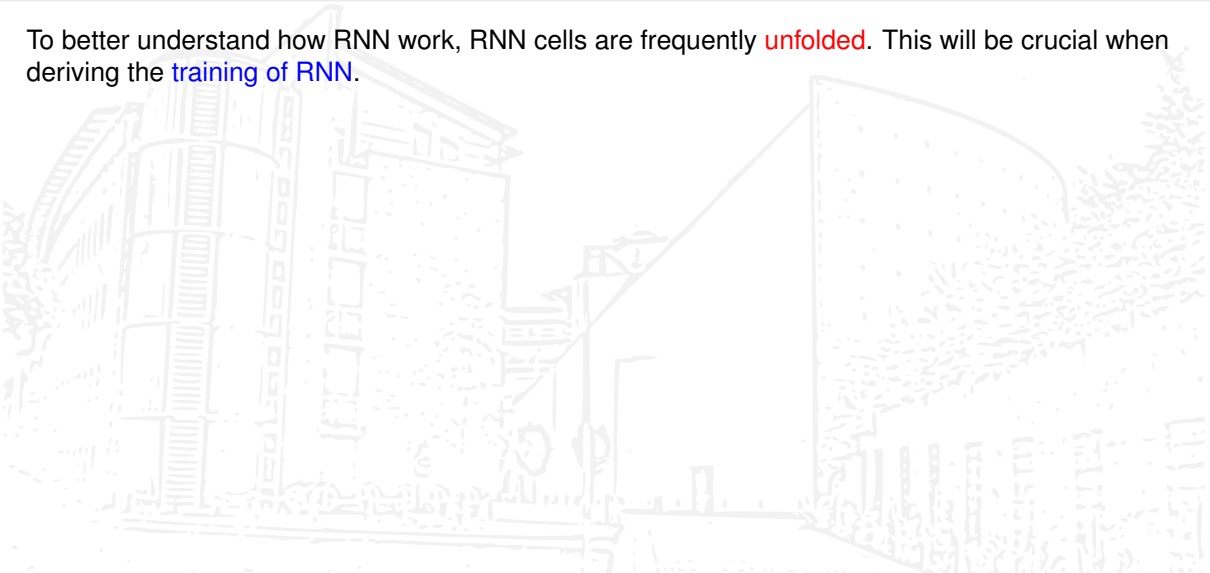
In **most applications**, the cell state is **internally completely described by**:

- ▶ **fixed data**, e.g., some fixed matrices, and
- ▶ the **variable update vector** that is handed to the next iteration, the **hidden state**.

Thus, RNN can **store information** (“a program”).

Unfolding an RNN cell

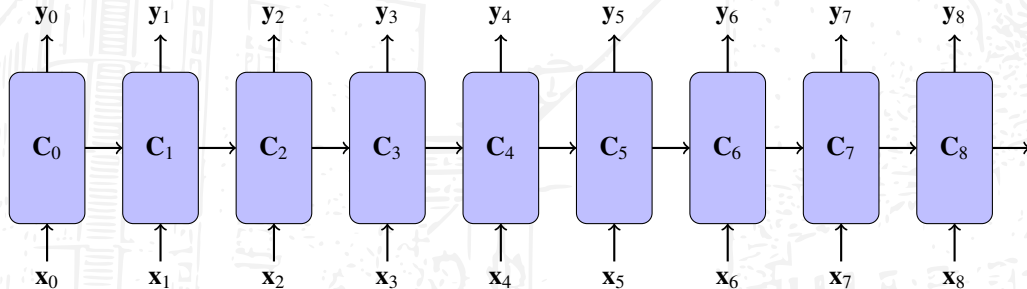
To better understand how RNN work, RNN cells are frequently **unfolded**. This will be crucial when deriving the **training of RNN**.



Unfolding an RNN cell

To better understand how RNN work, RNN cells are frequently **unfolded**. This will be crucial when deriving the **training of RNN**.

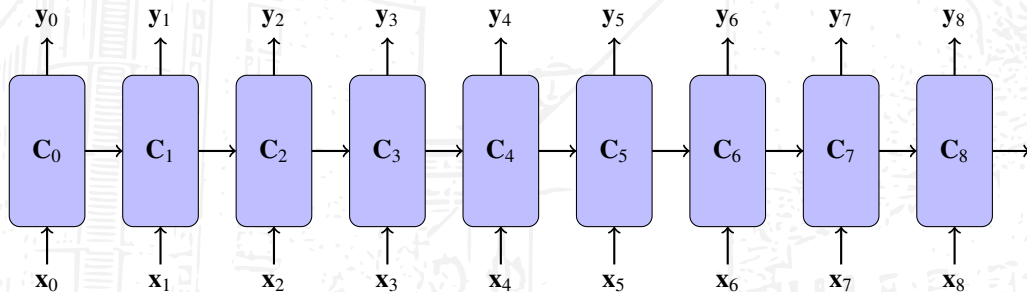
Example: an RNN cell **unfolded for 9 steps**



Unfolding an RNN cell

To better understand how RNN work, RNN cells are frequently **unfolded**. This will be crucial when deriving the **training of RNN**.

Example: an RNN cell **unfolded for 9 steps**



Thus, an RNN can be considered as a neural network that **shares weights over time**.

Example: generating sequences of numbers

Consider that we have an **integer sequence** $x_t \in \mathbb{N}_0$ as **input**, an **integer sequence** $y_t \in \mathbb{N}_0$ as **output**, and that the hidden state h_t ($h_{-1} = 0$) is equal to the output y_t , which is computed as

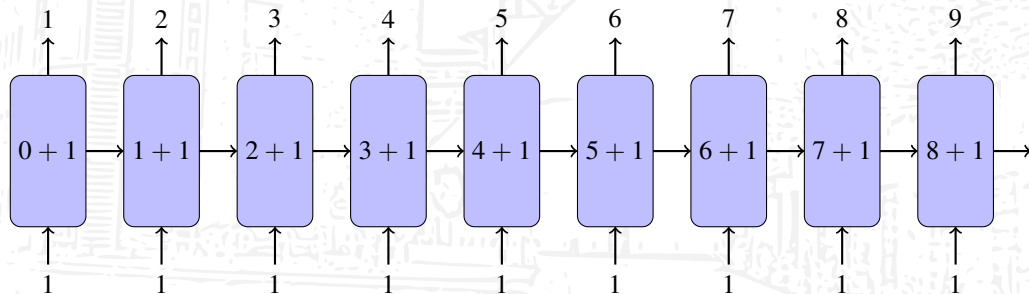
$$y_t = h_{t-1} + x_t = y_{t-1} + x_t = \sum_{k=0}^t x_k.$$

Example: generating sequences of numbers

Consider that we have an **integer sequence** $x_t \in \mathbb{N}_0$ as **input**, an **integer sequence** $y_t \in \mathbb{N}_0$ as **output**, and that the hidden state h_t ($h_{-1} = 0$) is equal to the output y_t , which is computed as

$$y_t = h_{t-1} + x_t = y_{t-1} + x_t = \sum_{k=0}^t x_k.$$

Then the RNN with initial hidden state $h_{-1} = 0$ computes the **cumulative sum**, e.g., the input 1, 1, 1, 1, ... generates the **natural numbers**:



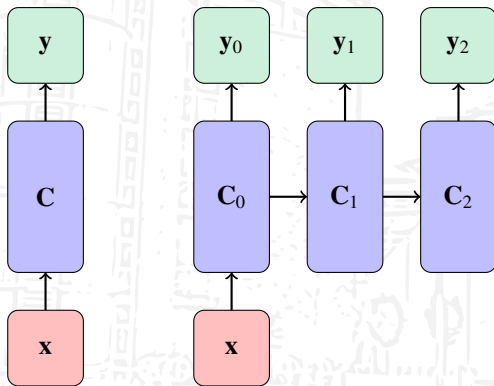
One to one

RNN come in many shapes: **one to one** (classical **feedforward neural network**),



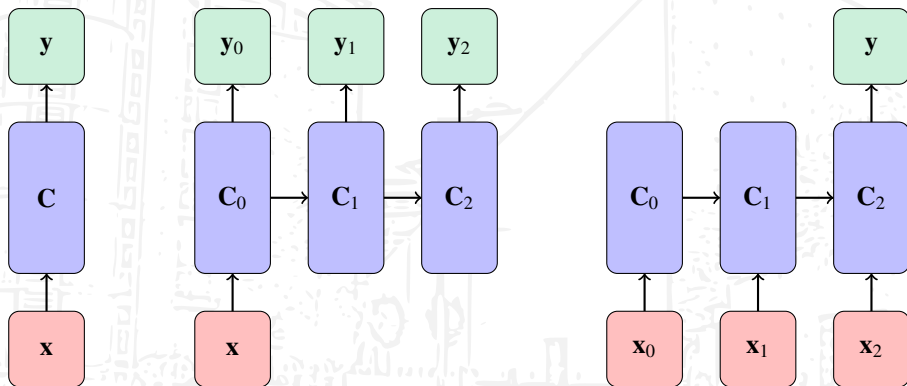
One to one, one to many

RNN come in many shapes: **one to one** (classical **feedforward neural network**), **one to many** (**image to caption**, “a picture is worth a thousand words”),



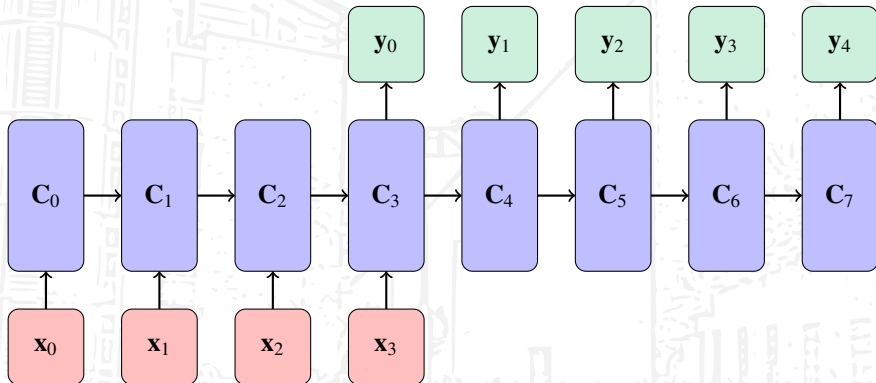
One to one, one to many, many to one

RNN come in many shapes: **one to one** (classical **feedforward neural network**), **one to many** (**image to caption**, “a picture is worth a thousand words”), **many to one** (e.g., **sentiment analysis**, decision problems, classification of input of variable length/size):



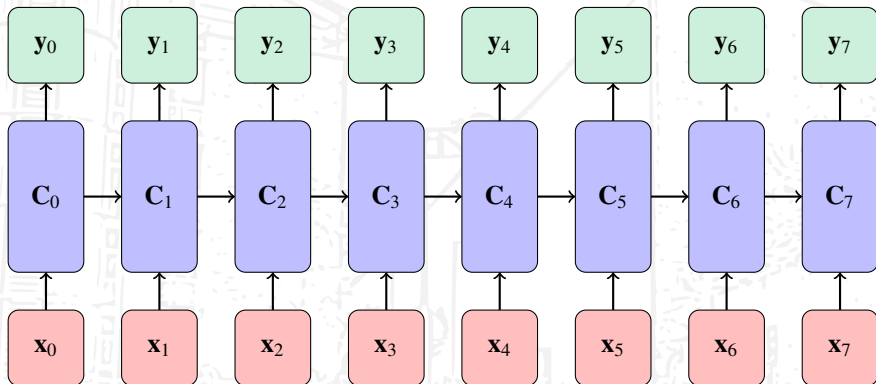
Many to many (first variant)

Many to many (delayed, e.g., **text translation**, texts in different languages are of different lengths: say “Hello!” to older versions of Siri, Alexa, Google translate):



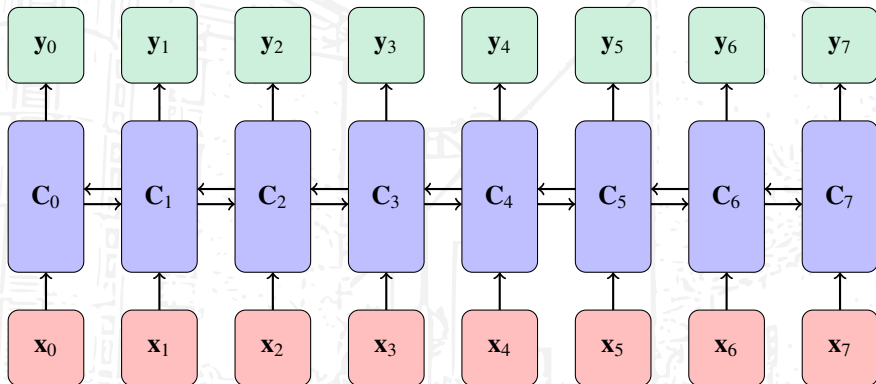
Many to many (second variant)

Many to many (synced, e.g., classification of video frames; **detection of objects in autonomous driving**; automated video editing; medical image registration):

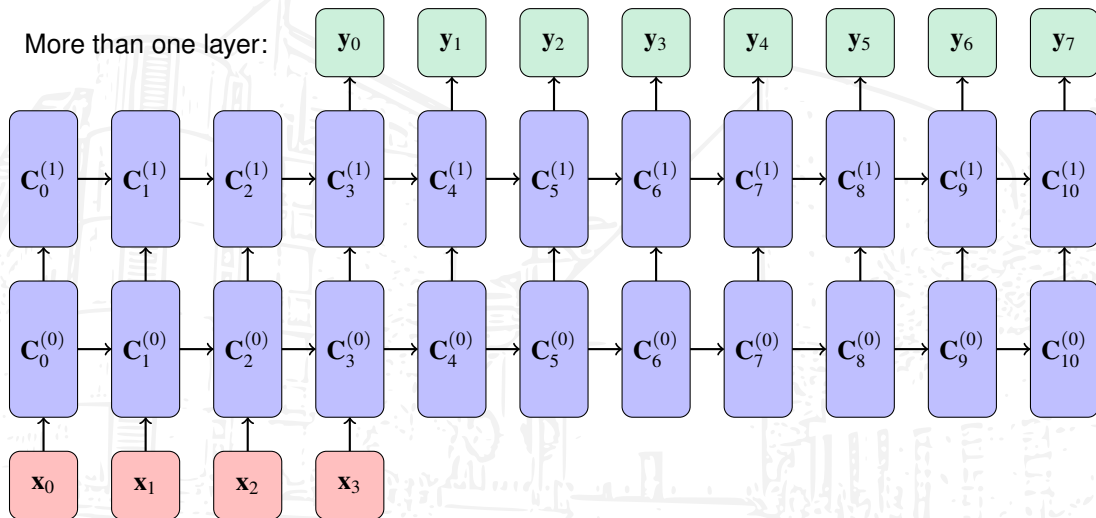


Bidirectional

Bidirectional, i.e., when the sequence is completely known, like a sentence to be translated, implemented as **two unidirectional recurrent layers** added/averaged/concatenated:



Several RNN layers



A definition of RNN

A recurrent neural network is a neural network which has (a) **feedback connection(s)**, e.g.,

- ▶ attached to a **single neuron** (like in the previous examples),
- ▶ between two neurons **in one layer**,
- ▶ from a neuron in one layer **to a neuron in a previous layer**.

Such networks exhibit **temporal behaviour**.

A definition of RNN

A recurrent neural network is a neural network which has (a) **feedback connection(s)**, e.g.,

- ▶ attached to a **single neuron** (like in the previous examples),
- ▶ between two neurons **in one layer**,
- ▶ from a neuron in one layer **to a neuron in a previous layer**.

Such networks exhibit **temporal behaviour**.

This implies that such networks can be interpreted as a **directed graph**, which may be either **cyclic** or **acyclic**. Thus far, we only treated the simpler case of **directed acyclic graphs**, as these can be **unrolled**. These are referred to as **finite impulse** recurrent neural networks. $\rightsquigarrow \text{RNN} \supseteq \text{FNN/CNN}$

A definition of RNN

A recurrent neural network is a neural network which has (a) **feedback connection(s)**, e.g.,

- ▶ attached to a **single neuron** (like in the previous examples),
- ▶ between two neurons **in one layer**,
- ▶ from a neuron in one layer **to a neuron in a previous layer**.

Such networks exhibit **temporal behaviour**.

This implies that such networks can be interpreted as a **directed graph**, which may be either **cyclic** or **acyclic**. Thus far, we only treated the simpler case of **directed acyclic graphs**, as these can be **unrolled**. These are referred to as **finite impulse** recurrent neural networks. $\leadsto \text{RNN} \supseteq \text{FNN/CNN}$

A warning: recurrent neural networks are special types of “**recursive neural networks**”, also abbreviated as RNN. Such a recursive NN is based on a **structure**, e.g., a (binary) tree. Recurrent NN have as structure the **linear structure**.

Simple RNN

Particular simple RNN are the so-called **Jordan** and **Elman** networks:



Simple RNN

Particular simple RNN are the so-called **Jordan** and **Elman** networks:

A **Jordan network** couples the hidden state with the previous **output** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hy}\mathbf{y}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0,$$

Simple RNN

Particular simple RNN are the so-called **Jordan** and **Elman** networks:

A **Jordan network** couples the hidden state with the previous **output** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hy}\mathbf{y}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0,$$

an **Elman network** couples the hidden state with the previous **hidden state** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

Simple RNN

Particular simple RNN are the so-called **Jordan** and **Elman** networks:

A **Jordan network** couples the hidden state with the previous **output** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hy}\mathbf{y}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0,$$

an **Elman network** couples the hidden state with the previous **hidden state** and is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

Initial output/state $\mathbf{y}_{-1} = \mathbf{h}_{-1} = \mathbf{o}$. Sizes of the vectors: $\mathbf{x}_t \in \mathbb{R}^{n_i}$, $\mathbf{h}_t \in \mathbb{R}^{n_h}$, $\mathbf{y}_t \in \mathbb{R}^{n_o}$.

Defining parameters:

- ▶ **weight matrices:** $\mathbf{W}_{hy} \in \mathbb{R}^{n_h \times n_o}$, $\mathbf{W}_{hh} \in \mathbb{R}^{n_h \times n_h}$, $\mathbf{W}_{hx} \in \mathbb{R}^{n_h \times n_i}$, and $\mathbf{W}_{yh} \in \mathbb{R}^{n_o \times n_h}$,
- ▶ **biases:** $\mathbf{b}_h \in \mathbb{R}^{n_h}$ and $\mathbf{b}_y \in \mathbb{R}^{n_o}$,
- ▶ **activation functions:** a_h for the hidden state (default: TanH), a_y for the output (e.g., SoftMax).

Our test example

We focus on an **Elman network**, our **vanilla RNN cell** is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

Our test example

We focus on an **Elman network**, our **vanilla RNN cell** is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

It can be shown that this class is **universal**: Any **Turing-computable** function can be computed by such a network of finite size. Feedforward neural nets: **optimization over functions**, RNN: **optimization over programs**.

Our test example

We focus on an **Elman network**, our **vanilla RNN cell** is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

It can be shown that this class is **universal**: Any **Turing-computable** function can be computed by such a network of finite size. Feedforward neural nets: **optimization over functions**, RNN: **optimization over programs**.

Implementation ($t \in \mathbb{N}_0$):

- ▶ **Class** `VanillaRNN(Layer)` : implements $\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$
- ▶ **Class** `SoftMaxRNN(Layer)` : implements $\mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y)$
- ▶ **Class** `Recurrent()` : at least one `VanillaRNN`, one `SoftMaxRNN`

Our test example

We focus on an **Elman network**, our **vanilla RNN cell** is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

It can be shown that this class is **universal**: Any **Turing-computable** function can be computed by such a network of finite size. Feedforward neural nets: **optimization over functions**, RNN: **optimization over programs**.

Implementation ($t \in \mathbb{N}_0$):

- ▶ Class `VanillaRNN(Layer)` : implements $\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$
- ▶ Class `SoftMaxRNN(Layer)` : implements $\mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y)$
- ▶ Class `Recurrent()` : at least one `VanillaRNN`, one `SoftMaxRNN`

Goal: RNN that maps **sequences of letters to the next letter**. “Hello” = “H”, “e”, “l”, “l”, “o” gives output “e”, “l”, “l”, “o” and an arbitrary letter.

Our test example

We focus on an **Elman network**, our **vanilla RNN cell** is based on

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0.$$

It can be shown that this class is **universal**: Any **Turing-computable** function can be computed by such a network of finite size. Feedforward neural nets: **optimization over functions**, RNN: **optimization over programs**.

Implementation ($t \in \mathbb{N}_0$):

- ▶ Class `VanillaRNN(Layer)` : implements $\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h)$
- ▶ Class `SoftMaxRNN(Layer)` : implements $\mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y)$
- ▶ Class `Recurrent()` : at least one `VanillaRNN`, one `SoftMaxRNN`

Goal: RNN that maps **sequences of letters to the next letter**. “Hello” = “H”, “e”, “l”, “l”, “o” gives output “e”, “l”, “l”, “o” and an arbitrary letter.

Testing: insert some letter, **use output as next input** etc. to generate sample.

Dynamical systems

RNN can be interpreted as **discrete dynamical systems** driven by some discrete input $\mathbf{x}_t$ and depending on parameters $\mathbf{p}$,

$$\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \mathbf{p}), \quad t \in \mathbb{N}_0.$$

Dynamical systems

RNN can be interpreted as **discrete dynamical systems** driven by some discrete input $\mathbf{x}_t$ and depending on parameters $\mathbf{p}$,

$$\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \mathbf{p}), \quad t \in \mathbb{N}_0.$$

We are interested in **many** ($\approx \infty$) **steps** of an RNN. If we neglect the input, the **stability of fixed points** $\mathbf{h}$ of the autonomous discrete system $\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1})$, $t \in \mathbb{N}_0$, defined by

- ▶ **$\mathbf{h}$ locally asymptotically stable** : $\Leftrightarrow$ there exists $U \ni \mathbf{h}$ such that for all $\mathbf{h}_{-1} \in U$ $\lim_{t \rightarrow \infty} \mathbf{h}_t = \mathbf{h}$,
- ▶ **$\mathbf{h}$ unstable** : $\Leftrightarrow \mathbf{h}$ is not locally asymptotically stable,

Dynamical systems

RNN can be interpreted as **discrete dynamical systems** driven by some discrete input $\mathbf{x}_t$ and depending on parameters $\mathbf{p}$,

$$\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t, \mathbf{p}), \quad t \in \mathbb{N}_0.$$

We are interested in **many** ($\approx \infty$) **steps** of an RNN. If we neglect the input, the **stability of fixed points** $\mathbf{h}$ of the autonomous discrete system $\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1})$, $t \in \mathbb{N}_0$, defined by

- ▶ **$\mathbf{h}$ locally asymptotically stable** $:\Leftrightarrow$ there exists $U \ni \mathbf{h}$ such that for all $\mathbf{h}_{-1} \in U$ $\lim_{t \rightarrow \infty} \mathbf{h}_t = \mathbf{h}$,
- ▶ **$\mathbf{h}$ unstable** $:\Leftrightarrow \mathbf{h}$ is not locally asymptotically stable,

is related to the **eigenvalues of the Jacobian** $\mathbf{J}(\mathbf{h})$ of $\mathbf{f} \in C^2$ at $\mathbf{h}$:

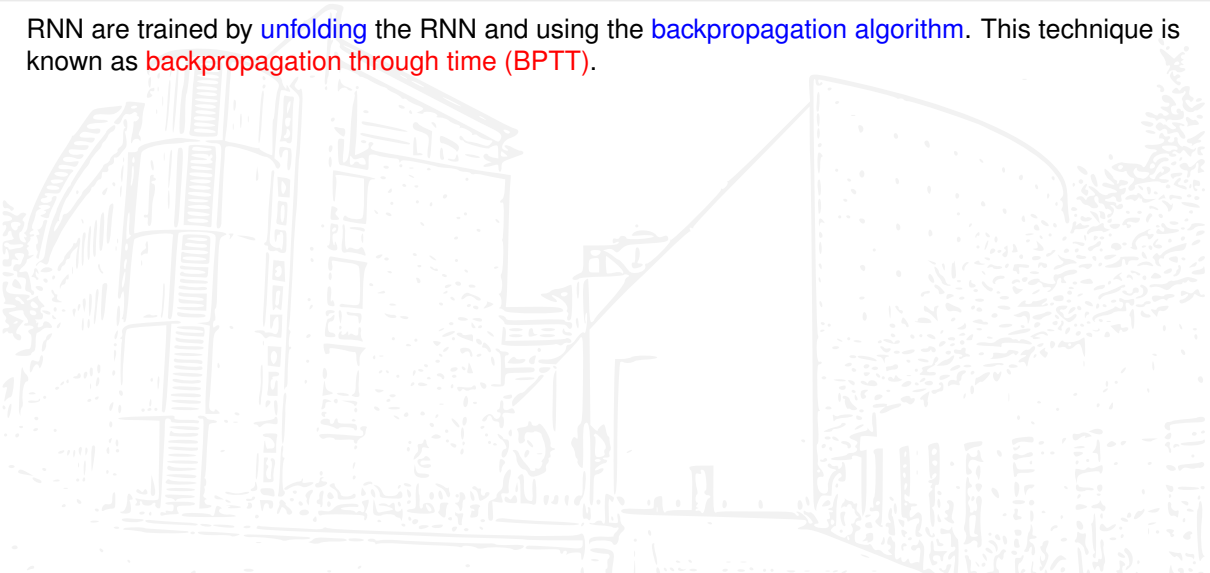
- ▶ $\rho(\mathbf{J}(\mathbf{h})) < 1 \Rightarrow \mathbf{h}$ locally asymptotically stable,
- ▶ $\rho(\mathbf{J}(\mathbf{h})) > 1 \Rightarrow \mathbf{h}$ unstable.

Here, ρ denotes the **spectral radius**

$$\rho(\mathbf{A}) := \max\{|\lambda| \mid \lambda \text{ is an eigenvalue of } \mathbf{A}\}.$$

Backpropagation through time

RNN are trained by **unfolding** the RNN and using the **backpropagation algorithm**. This technique is known as **backpropagation through time (BPTT)**.



Backpropagation through time

RNN are trained by **unfolding** the RNN and using the **backpropagation algorithm**. This technique is known as **backpropagation through time (BPTT)**. We need the **derivatives of all states** for every time t for **finite sequences**, as we can only use those during training due to memory restrictions.

Backpropagation through time

RNN are trained by **unfolding** the RNN and using the **backpropagation algorithm**. This technique is known as **backpropagation through time (BPTT)**. We need the **derivatives of all states** for every time t for **finite sequences**, as we can only use those during training due to memory restrictions.

We consider an **Elman RNN**

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

unfolded for $k + 1$ steps, **two finite sequences** $\{\mathbf{x}_t\}_{t=0}^k$ and $\{\mathbf{y}_t\}_{t=0}^k$, and a **cost function**

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

Backpropagation through time

RNN are trained by **unfolding** the RNN and using the **backpropagation algorithm**. This technique is known as **backpropagation through time (BPTT)**. We need the **derivatives of all states** for every time t for **finite sequences**, as we can only use those during training due to memory restrictions.

We consider an **Elman RNN**

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

unfolded for $k + 1$ steps, **two finite sequences** $\{\mathbf{x}_t\}_{t=0}^k$ and $\{\mathbf{y}_t\}_{t=0}^k$, and a **cost function**

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t).$$

Similar to the case of the **feedforward net** we can compute for all $t = 0, \dots, k$

$$\frac{\partial C}{\partial C_t} = 1, \quad \frac{\partial C}{\partial \mathbf{y}_t} = \frac{\partial C_t}{\partial \mathbf{y}_t}, \quad \frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k} = \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{W}_{yh}\mathbf{h}_k + \mathbf{b}_y) \circ \frac{\partial C_k}{\partial \mathbf{y}_k} \right).$$

Backpropagation through time

The **Elman RNN** is given by

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

Backpropagation through time

The **Elman RNN** is given by

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

To **simplify notation**, we define

$$\mathbf{a}_t := \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h, \quad \mathbf{z}_t := \mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y, \quad t \in \mathbb{N}_0.$$

Backpropagation through time

The **Elman RNN** is given by

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

To **simplify notation**, we define

$$\mathbf{a}_t := \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h, \quad \mathbf{z}_t := \mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y, \quad t \in \mathbb{N}_0.$$

Thus, the **derivative of C** with respect to the **final $\mathbf{h}_k$** is given by

$$\frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k} = \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_k) \circ \frac{\partial C_k}{\partial \mathbf{y}_k} \right).$$

Backpropagation through time

The **Elman RNN** is given by

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

To **simplify notation**, we define

$$\mathbf{a}_t := \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h, \quad \mathbf{z}_t := \mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y, \quad t \in \mathbb{N}_0.$$

Thus, the **derivative of C** with respect to the **final $\mathbf{h}_k$** is given by

$$\frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k} = \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_k) \circ \frac{\partial C_k}{\partial \mathbf{y}_k} \right).$$

The derivative with respect to the **previous $\mathbf{h}_t$** **additionally depends on $\mathbf{h}_{t+1}$** , $0 \leq t < k$,

$$\frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \sum_{\ell > t} C_\ell}{\partial \mathbf{h}_t} + \frac{\partial C_t}{\partial \mathbf{h}_t} = \mathbf{W}_{hh}^\top \left(a'_h(\mathbf{a}_{t+1}) \circ \frac{\partial \sum_{\ell > t} C_\ell}{\partial \mathbf{h}_{t+1}} \right) + \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_t) \circ \frac{\partial C_t}{\partial \mathbf{y}_t} \right).$$

Backpropagation through time

The **Elman RNN** is given by

$$\mathbf{h}_t = a_h(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = a_y(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y), \quad t \in \mathbb{N}_0$$

To **simplify notation**, we define

$$\mathbf{a}_t := \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{b}_h, \quad \mathbf{z}_t := \mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y, \quad t \in \mathbb{N}_0.$$

Thus, the **derivative of C** with respect to the **final $\mathbf{h}_k$** is given by

$$\frac{\partial C}{\partial \mathbf{h}_k} = \frac{\partial C_k}{\partial \mathbf{h}_k} = \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_k) \circ \frac{\partial C_k}{\partial \mathbf{y}_k} \right).$$

The derivative with respect to the **previous $\mathbf{h}_t$** **additionally depends on $\mathbf{h}_{t+1}$** , $0 \leq t < k$,

$$\frac{\partial C}{\partial \mathbf{h}_t} = \frac{\partial \sum_{\ell > t} C_\ell}{\partial \mathbf{h}_t} + \frac{\partial C_t}{\partial \mathbf{h}_t} = \mathbf{W}_{hh}^\top \left(a'_h(\mathbf{a}_{t+1}) \circ \frac{\partial C}{\partial \mathbf{h}_{t+1}} \right) + \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_t) \circ \frac{\partial C_t}{\partial \mathbf{y}_t} \right).$$

Backpropagation through time

Theorem (BPTT for Elman RNN)

Let an Elman RNN unfolded for $k + 1$ steps, two finite sequences $\{\mathbf{x}_t\}_{t=0}^k$ and $\{\mathbf{y}_t\}_{t=0}^k$, and a cost function

$$C(\{\mathbf{x}_t\}_{t=0}^k, \{\mathbf{y}_t\}_{t=0}^k) := \sum_{t=0}^k C_t(\{\mathbf{x}_\ell\}_{\ell=0}^t, \mathbf{y}_t)$$

be given. Then the derivatives with respect to the hidden states can be computed recursively starting at the end time $t = k$ and proceeding backwards through time, $0 \leq t < k$,

$$\frac{\partial C}{\partial \mathbf{h}_t} = \mathbf{W}_{hh}^\top \left(a'_h(\mathbf{a}_{t+1}) \circ \frac{\partial C}{\partial \mathbf{h}_{t+1}} \right) + \mathbf{W}_{yh}^\top \left(a'_y(\mathbf{z}_t) \circ \frac{\partial C}{\partial \mathbf{y}_t} \right),$$

where in case $t = k$ the terms involving $t + 1$ are omitted.

Backpropagation through time

Theorem (BPTT for Elman RNN II)

The derivatives of the cost function with respect to the two biases, three weight matrices, and $k + 1$ inputs are given by

$$\frac{\partial C}{\partial \mathbf{b}_y} = \sum_{t=0}^k a'_y(\mathbf{z}_t) \circ \frac{\partial C}{\partial \mathbf{y}_t},$$

$$\frac{\partial C}{\partial \mathbf{b}_h} = \sum_{t=0}^k a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t},$$

$$\frac{\partial C}{\partial \mathbf{W}_{yh}} = \sum_{t=0}^k \left(a'_y(\mathbf{z}_t) \circ \frac{\partial C}{\partial \mathbf{y}_t} \right) \mathbf{h}_t^\top,$$

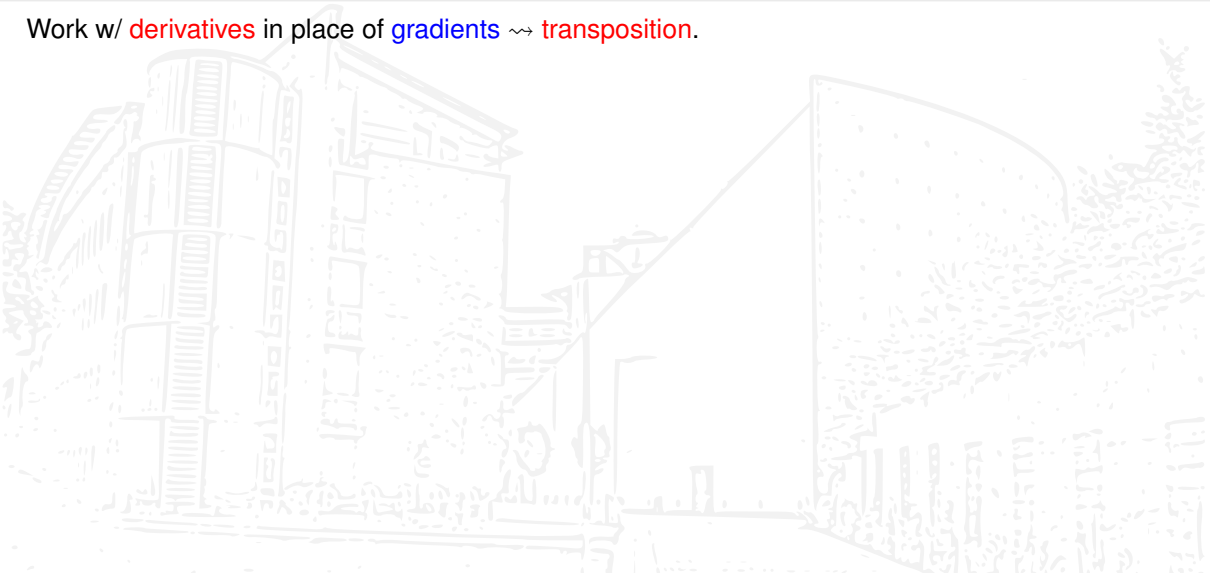
$$\frac{\partial C}{\partial \mathbf{W}_{hh}} = \sum_{t=0}^k \left(a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t} \right) \mathbf{h}_{t-1}^\top,$$

$$\frac{\partial C}{\partial \mathbf{W}_{hx}} = \sum_{t=0}^k \left(a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t} \right) \mathbf{x}_t^\top,$$

$$\frac{\partial C}{\partial \mathbf{x}_t} = \mathbf{W}_{hx}^\top \left(a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t} \right), \quad t = 0, \dots, k.$$

A few implementation details

Work w/ **derivatives** in place of **gradients** $\rightsquigarrow$ **transposition**.



A few implementation details

Work w/ **derivatives** in place of **gradients** $\rightsquigarrow$ **transposition**.

Feedforward: In `layer = VanillaRNN()`, `y = layer.feedforward(x)` compute

$$\mathbf{h}_t = \text{TanH}(\mathbf{h}_{t-1}\mathbf{W}_h + \mathbf{x}_t\mathbf{W}_x + \mathbf{b}),$$

store `x` in `self.x`, in `self.y` **extended output** `self.y[:,t,:] =  $\mathbf{h}_{t-1}$` , return `self.y[:, 1:, :]`.

A few implementation details

Work w/ **derivatives** in place of **gradients** $\rightsquigarrow$ **transposition**.

Feedforward: In `layer = VanillaRNN()`, `y = layer.feedforward(x)` compute

$$\mathbf{h}_t = \text{TanH}(\mathbf{h}_{t-1}\mathbf{W}_h + \mathbf{x}_t\mathbf{W}_x + \mathbf{b}),$$

store `x` in `self.x`, in `self.y` **extended output** `self.y[:,t,:] =  $\mathbf{h}_{t-1}$` , return `self.y[:, 1:, :]`.

Backpropagation: in `dx = layer.backprop(dy)` set and compute **recursively for all t**

$$da := a'_h(\mathbf{a}_t) \circ \frac{\partial C}{\partial \mathbf{h}_t},$$

then **updates for** derivatives of cost function w.r.t. **matrices similar:**

```
self.dWh += self.y[:,t,:].T @ da,
self.dWx += self.x[:,t,:].T @ da.
```

References I

- [1] C. G. Broyden. “The convergence of a class of double-rank minimization algorithms. II. The new algorithm”. In: *J. Inst. Math. Appl.* 6 (1970), pp. 222–231. ISSN: 0020-2932. DOI: [10.1093/imamat/6.1.76](https://doi.org/10.1093/imamat/6.1.76).
- [2] R. Fletcher. “A New Approach to Variable Metric Algorithms”. In: *Computer Journal* 13.3 (1970), pp. 317–322. DOI: [10.1093/comjnl/13.3.317](https://doi.org/10.1093/comjnl/13.3.317).
- [3] Donald Goldfarb. “A family of variable-metric methods derived by variational means”. In: *Math. Comp.* 24 (1970), pp. 23–26. ISSN: 0025-5718. DOI: [10.2307/2004873](https://doi.org/10.2307/2004873). URL: <https://doi.org/10.2307/2004873>.
- [4] I. J. Goodfellow, J. Shlens, and C. Szegedy. “Explaining and harnessing adversarial examples”. In: *arXiv e-prints*, arXiv:1412.6572 (Dec. 2014). arXiv: 1412.6572 [cs.LG].
- [5] Jorge Nocedal. “Updating quasi-Newton matrices with limited storage”. In: *Math. Comp.* 35.151 (1980), pp. 773–782. ISSN: 0025-5718. DOI: [10.2307/2006193](https://doi.org/10.2307/2006193). URL: <https://doi.org/10.2307/2006193>.

References II

- [6] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. “Feature Visualization”. In: *Distill* (2017). DOI: 10.23915/distill.00007. URL: <https://distill.pub/2017/feature-visualization>.
- [7] D. F. Shanno. “Conditioning of quasi-Newton methods for function minimization”. In: *Math. Comp.* 24 (1970), pp. 647–656. ISSN: 0025-5718. DOI: 10.2307/2004840. URL: <https://doi.org/10.2307/2004840>.
- [8] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. “Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps”. In: *arXiv e-prints*, arXiv:1312.6034 (Dec. 2013). arXiv: 1312.6034 [cs.CV].
- [9] C. Szegedy et al. “Intriguing properties of neural networks”. In: *arXiv e-prints*, arXiv:1312.6199 (Dec. 2013). arXiv: 1312.6199 [cs.CV].